DuetScale: Scaling Disaggregated LLM Serving with Joint Parallelism and Startup Optimization

Yihao Zheng
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
225040496@link.cuhk.edu.cn

Zhiyuan Tan
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
zhiyuantan1@link.cuhk.edu.cn

Yicheng Huo
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
huohuoyicheng@gmail.com

Bohui Wu
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
225010300@link.cuhk.edu.cn

Yiyun Zheng
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
yiyunzheng@link.cuhk.edu.cn

Minchen Yu*
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
yuminchen@cuhk.edu.cn

Abstract

Dynamic LLM workloads vary in request concurrency and sequence length, changing both GPU resource demand and how those resources should be organized through parallelism. Existing autoscaling policies primarily focus on how much serving capacity to provision, while the parallelism configuration of newly added resources is often predetermined. A fixed configuration can therefore mismatch the workload, while instance startup delays can make an otherwise suitable choice take effect too late. We present DuetScale, a study of these coupled configuration and timing mismatches in disaggregated LLM serving. DuetScale characterizes TP, PP, and DP under an equal GPU budget and develops white-box models for TTFT, TPOT, and throughput across workload states. We also develop a parallel-instance startup optimization that reuses distributed execution state. Together, these contributions provide the foundations for a workload-aware scaling policy that chooses the amount and organization of GPU resources for the target workload and makes them available in time. Building and evaluating that end-to-end policy remains future work.

CCS Concepts

• **Computer systems organization → Cloud computing**.

Keywords

large language model serving, autoscaling, disaggregated inference, parallelism

1 Introduction

LLM serving workloads exhibit substantial temporal variation in arrival token rate. Production traces show that request arrival rate and sequence length can vary independently over time, jointly determining the resulting token arrival rate [13, 15], as illustrated in Figure 1. Request arrival rate determines how frequently requests enter the system, whereas input and output lengths determine the amount of work per request in Prefill and Decode, respectively. The same aggregate token rate can therefore arise from many short requests or fewer long requests. These workload shapes expose different degrees of inter-request parallelism and impose different per-request computation and KV-cache demands.

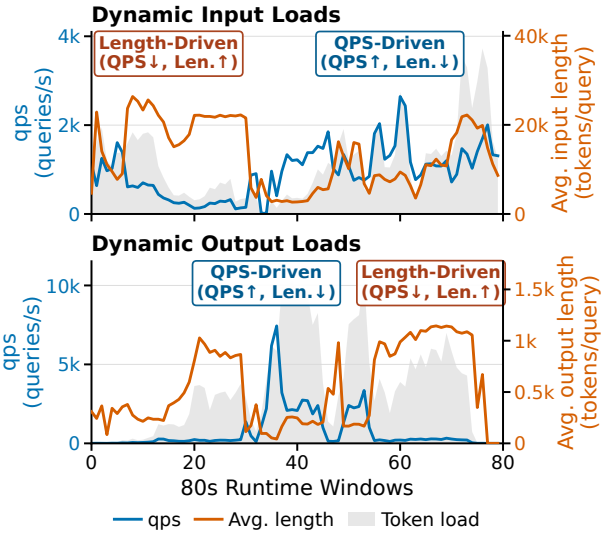


Figure 1: Dynamic BurstGPT workload: request arrival rate and sequence length jointly determine Prefill and Decode token load.

At a coarse granularity, changes in aggregate token rate determine how many GPUs the service needs. Provisioning for peak demand leaves GPUs idle during quiet periods, whereas provisioning for average demand allows queues to build during bursts. Autoscaling mitigates the trade-off between resource utilization and latency by adjusting the number of active GPUs in response to changes in the aggregate token rate.

The throughput delivered by additional resources, however, depends on both their quantity and their parallelism configuration.At each scale-out event, a controller must also choose a parallelism configuration for the added GPUs—for example, a TP or PP model-parallel instance, or a set of DP replicas. These configurations can provide different levels of throughput under the same GPU budget, and the configuration that achieves the highest throughput varies

*Corresponding author.

with request concurrency and sequence length.Matching the parallelism configuration to the workload is therefore necessary to obtain high per-GPU throughput from the added GPUs. A scaling policy that always adds the same configuration must either allocate extra GPUs to make up for its lower throughput or accept longer queues.

Configuration selection alone is not enough: timely instance startup is equally important. An appropriate configuration is useful only if it becomes available while the corresponding workload demand persists. A newly added model instance cannot serve requests until model loading and distributed runtime initialization complete. If concurrency or sequence length changes during this interval, an instance selected for the workload at decision time may no longer match the workload when it begins serving, leaving the service temporarily underprovisioned and allowing queues to grow. The instance and its target workload must therefore coexist in time before routing can realize the match; we leave this routing problem to future work. Effective scale-out therefore requires both a suitable parallelism configuration and a startup latency short enough for the new instance to serve the workload that motivated the decision.

Prior work has explored resource control and instance startup optimization, but their co-design has received limited attention. Resource control determines both how much capacity to add and which parallelism configuration to use. Instantiating the selected configuration requires creating distributed process groups and initializing communication channels. These steps increase instance startup latency and make startup optimization particularly important for timely scale-out.We therefore take a co-design approach to GPU-efficient dynamic scale-out, considering configuration performance together with the startup cost of making the selected configuration available. We characterize and model the throughput of different parallelism configurations under a fixed GPU budget.Complementing this analysis, WorkerPool reuses distributed execution state to reduce the delay before newly provisioned instances can serve requests.

This workshop paper makes the following contributions:

- **Parallelism analysis and modeling.** Under a fixed eight-GPU budget, we characterize how the performance of TP, PP, and DP changes with concurrency and sequence length in Prefill and Decode. We develop white-box models for TTFT, TPOT, and throughput that capture these trends and estimate configuration-specific performance across workload states.
- **Parallel-instance startup optimization.** We introduce *WorkerPool*, which keeps pre-initialized workers and reuses distributed execution state across scaling events. WorkerPool reduces measured TP4 instance startup latency from 51.9 seconds to 6.5 seconds, an 8.0× speedup.

2 Motivation and Background

Scaling in disaggregated serving. Disaggregated serving systems make it possible to provision Prefill and Decode independently [9, 10, 21]. Autoscaling and runtime systems use inference-aware signals and rebalance requests as load changes [1, 7, 13]. Together, these systems provide important foundations for scalable disaggregated serving. Existing autoscaling policies, however, generally emphasize when and how many GPUs to add, while the parallelism configuration of new instances is often predetermined. Consequently, how new instances should be configured to match the workload shape remains less explored. Parallel-instance startup has also received less attention in this configuration-selection context, even though a suitable instance helps only if it becomes available while the target workload persists. These are coupled questions of GPU-efficient scale-out, which we examine in turn.

Parallelism configuration. The request-rate- and length-driven intervals in Figure 1 show why aggregate token rate alone cannot determine the most effective parallelism configuration. LoongServe adjusts sequence parallelism across requests and phases, Seesaw reshards the model between Prefill and Decode, and Shift Parallelism switches between intra- and inter-request parallelism as traffic changes [4, 12, 16]. These systems reinforce the importance of matching parallelism to workload characteristics. They obtain this match by adapting execution within an active serving system. These approaches can add transition latency and other overheads, and focus on reshaping existing execution rather than adjusting the total GPU allocation. We study the complementary scale-out question of how added GPUs should be configured without changing running instances. This requires estimating which configuration provides the highest per-GPU throughput for the target workload.

Startup optimization. Prior startup optimizations target different parts of model startup. ServerlessLLM accelerates checkpoint loading and accounts for data locality, FaaScale overlaps model transfer with inference, and Foundry materializes CUDA graph state from reusable templates [3, 8, 19]. These techniques optimize important parts of model startup, while repeated initialization of a complete multi-rank inference runtime remains less explored. We therefore ask: which parts of distributed runtime state must be reconstructed for every scale-out event? A decision made for workload x_t takes effect only after the startup interval; if the workload changes on that time scale, the selected model instance may become available only after its target workload has changed.

3 Approach

To address the configuration and temporal mismatches identified above—two coupled aspects of GPU-efficient dynamic scale-out—we develop a configuration-specific performance model and implement a parallel-instance startup optimization. All experiments use Llama-3.1-70B-Instruct FP8 served by SGLang v0.5.9 on NVIDIA H20 GPUs.

3.1 Parallelism Analysis and Modeling

Problem formulation. At scale-out, the throughput provided by added GPUs depends on both workload shape and parallelism configuration. For workload state x, which captures request concurrency and the relevant sequence length, let $Q(x;\pi)$ denote the aggregate throughput of parallelism configuration π under a fixed GPU budget. We study how Q varies jointly with workload and parallelism configuration. If the ordering of $Q(x;\pi)$ across configurations changes with x, scale-out must match the configuration of the added instance to its target workload.

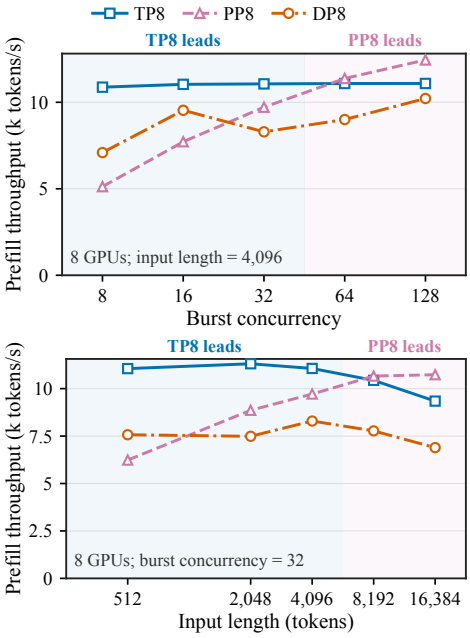


Figure 2: Prefill throughput under an equal eight-GPU budget across workload sweeps.

Empirical observations. To examine this relationship, we compare TP8, PP8, and DP8 under the same eight-GPU budget while varying concurrency and the phase-relevant sequence length. TP8 and PP8 each use one eight-GPU model-parallel instance, whereas DP8 uses eight independent one-GPU replicas.

For Prefill, Figure 2 shows that TP8 leads at concurrency 8–32 for an input length of 4,096, while PP8 overtakes it at concurrency 64–128. At concurrency 32, the crossover occurs as the input length increases from 4,096 to 8,192 tokens. TP efficiently accelerates each request when little inter-request parallelism is available, whereas greater concurrency or longer inputs provide enough chunks to keep PP stages busy and amortize pipeline bubbles.

Insight 1 (Prefill). Under the same GPU budget, TP provides higher throughput at light Prefill loads, whereas PP becomes preferable as request concurrency or input length increases.

For Decode, Figure 3 shows that TP8 leads through concurrency 128, while DP8 provides 31.2% higher throughput at concurrency 256. In the output-length sweep at fixed concurrency, TP8 remains preferable, although increasing context length raises KV-cache movement and produces a moderate throughput decline. TP reduces the latency of each decoding step when few requests are available, whereas sufficient independent requests allow DP replicas to run concurrently.

Insight 2 (Decode). Under the same GPU budget, TP provides higher throughput at low Decode concurrency, whereas DP becomes preferable as concurrency increases.

These observations confirm that $Q(x;\pi)$ is workload dependent: no single configuration maintains the highest throughput across the measured workload states. Configuration selection therefore requires predicting the throughput of each candidate configuration for the target workload.

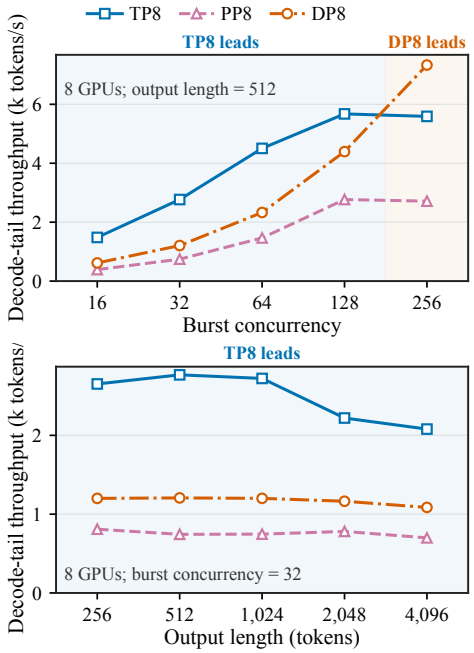


Figure 3: Decode throughput under an equal eight-GPU budget across workload sweeps.

Performance modeling. We model the latency components that determine throughput rather than assigning a fixed throughput value to each parallelism configuration. Let $\pi = (R, L, T)$ denote the DP replica count, PP depth, and TP degree under a fixed GPU budget. The execution time of one forward pass under workload state x is

$$\tau(x;\pi) = \max\big(T_{\text{comp}}(x;\pi), T_{\text{mem}}(x;\pi)\big) + T_{\text{comm}}(x;\pi). \quad (1)$$

The three terms represent computation, memory movement (including model weights and the KV cache), and communication for both phases. We use the basic form of Little's law for queueing time:

$$W_q = \frac{L_q}{\lambda}. \quad (2)$$

For Prefill, $x_r = m_r$ is the input chunk size in round r. If K_p rounds execute before the first token, the request-level latency is

$$\widehat{\text{TTFT}}_{50} = W_q + \sum_{r=1}^{K_p} \tau(x_r;\pi). \quad (3)$$

For S_Σ processed prompt tokens and total Prefill time T_p, aggregate throughput is

$$\widehat{Q}_p = \frac{S_\Sigma}{T_p}. \quad (4)$$

At Decode step t, let $x_{t,j} = (b_{t,j}, \ell_t)$ describe the batch size and context length of forward pass j. With J_t forward passes, per-token latency is

$$\widehat{\mathrm{TPOT}}_{50} = W_q + \frac{1}{S-1} \sum_{t=1}^{S-1} \sum_{j=1}^{J_t} \tau(x_{t,j}; \pi). \tag{5}$$

For C queued requests, output length S, and total Decode time T_d, throughput is

$$\widehat{Q}_d = \frac{C(S-1)}{T_d}. \tag{6}$$

Figure 4 evaluates the shared parameterization across hundreds of TP8, PP8, and DP8 workload settings. The median relative errors are 9.45% for Prefill TTFT and 5.59% for Decode TPOT. Together with the throughput definitions above, the same model also captures the configuration-specific throughput trends in Figures 2 and 3, providing accurate estimates of TTFT, TPOT, and throughput across all three parallelism configurations. The cost decomposition further explains the two insights. At light Prefill load, the reduction in TP's per-request computation and memory time outweighs its communication overhead; as concurrency or input length grows, greater pipeline utilization amortizes PP's pipeline bubbles. For Decode, TP's shorter per-step execution time dominates at low concurrency, whereas independent DP replicas distribute queued requests at high concurrency, reducing W_q enough to overtake TP. Longer Decode contexts also increase T_{mem}, accounting for the moderate output-length decline. Given a target workload and its throughput shortfall, the model can rank candidate configurations by per-GPU throughput and estimate how many instances of the selected configuration are needed, informing both the amount and organization of resources added during scale-out. The fitted model can also support throughput-crossover prediction, with median errors of 7.1% for Prefill and 5.1% for Decode.

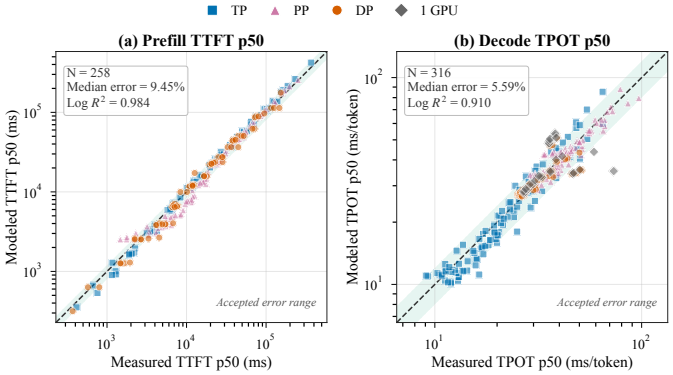


Figure 4: Measured and modeled Prefill TTFT and Decode TPOT; shading marks the accepted ±20% relative-error range.

3.2 Parallel-Instance Startup Optimization

A scale-out decision made from workload state x_t can affect service only at $t + T_{\mathrm{start}}$. Startup is therefore an actuation delay of the scaling loop, not merely deployment overhead. As this delay grows relative to the rate of workload change, the configuration selected at t is more likely to be stale when it begins serving.

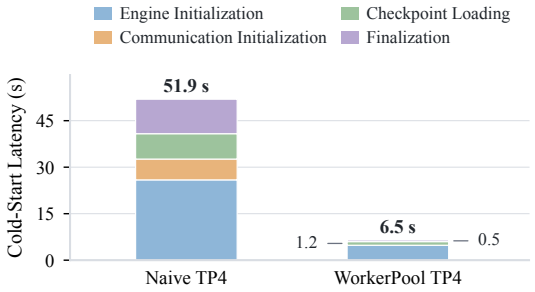


Figure 5: Startup latency breakdown for naive SGLang and WorkerPool TP4 on H20 GPUs.

Figure 5 shows that engine and communication initialization dominate naive SGLang TP4 startup. These phases repeatedly reconstruct rank assignments and communication contexts that can remain stable across scaling events.

Insight 3 (Startup). Distributed state that remains stable across scaling events can be reused to remove repeated initialization from the critical path of parallel-instance startup.

A parallel instance can serve only after engine initialization, communication setup, model loading, and finalization. For configuration π, its startup latency is

$$T_{\mathrm{start}}(\pi) = \underbrace{T_{\mathrm{eng}}(\pi) + T_{\mathrm{comm}}(\pi)}_{\text{reusable initialization}} + \underbrace{T_{\mathrm{load}}(\pi) + T_{\mathrm{final}}(\pi)}_{\text{online activation}}. \tag{7}$$

The equation exposes the optimization opportunity: engine and communication state that remains invariant across scale-out events can be retained, leaving only the required model-state restoration and finalization on the online activation path.

WorkerPool initializes workers for candidate configurations in advance and keeps them available for reuse. Inactive workers retain their rank assignments and NCCL communication contexts; their rank-local model state is restored on activation. Scale-out therefore reuses initialized runtime state instead of reconstructing it, while preserving the restoration work required for the selected instance.

This optimization trades online startup time for bounded host-memory space. The checkpoint and warmed TP/PP rank-local states are cached in pinned shared host (CPU) memory and restored to GPUs when a slot is activated, rather than kept as persistent GPU copies. The footprint therefore grows with the retained profiles and model size and must be bounded by the pool design. WorkerPool also pre-creates the candidate groups at the first system startup, making that startup longer than starting a single native instance; later scale-outs amortize this one-time cost. The 6.5-second value in Figure 5 is consequently a post-warmup reactivation time, not the cost of constructing the initial pool.

Reusing this runtime state reduces startup latency from 51.9 to 6.5 seconds, an 8.0× speedup. More importantly for dynamic scaling, it shortens the interval over which the workload can diverge from the state that motivated the scale-out decision.

4 Future Work

SLO-aware scaling policy. We will combine the model and WorkerPool to balance throughput and latency. The Estimator assesses

dynamic demand; the Planner selects the fewest GPUs and an instance mix that meets TTFT and TPOT SLOs, targeting high per-GPU goodput:

$$\min_{\mathbf{n}} \sum_{\pi} g_\pi n_\pi \quad \text{s.t.} \quad \mathbf{n} \in \mathcal{F}_{\text{SLO}}(x), \tag{8}$$

where n_π and g_π denote the instance count and GPUs per instance. $\mathcal{F}_{\text{SLO}}(x)$ contains instance mixes that meet the throughput demand and tail-latency SLOs of workload x. Startup delays and demand uncertainty will inform these constraints.

Agentic inference autoscaling. For agentic workloads, latency objectives extend beyond per-call TTFT and TPOT to end-to-end task latency. We will investigate how task-level latency SLOs affect latency–throughput trade-offs in autoscaling. Despite their complexity, agentic workloads can exhibit recurring demand patterns driven by agent execution loops and tool calls. We will investigate how these patterns provide control signals for proactive scaling.

5 Conclusion

This paper explores GPU-efficient scale-out for dynamic, disaggregated LLM serving. We treat configuration and timing mismatches as two coupled aspects of the same problem: added GPUs must be organized for the target workload and become available while that workload persists. We characterize and model how parallelism performance changes with workload shape, and explore reusable distributed state as a way to shorten parallel-instance startup. These components do not yet constitute an evaluated end-to-end scaling policy. Future work will integrate them into an end-to-end workload-aware scaling policy, incorporate SLO-aware objectives, and extend the analysis to agentic inference workloads.

References

[1] AIBrix Team. 2025. AIBrix: Towards Scalable, Cost-Effective Large Language Model Inference Infrastructure. *arXiv preprint arXiv:2504.03648* (2025).

[2] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.

[3] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation*.

[4] Mert Hidayetoglu, Andrew Qiao, Michael Wyatt, Jeff Rasley, Yuxiong He, and Samyam Rajbhandari. 2025. Shift Parallelism: Low-Latency, High-Throughput LLM Inference for Dynamic Workloads. *arXiv preprint arXiv:2509.16495* (2025).

[5] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-Throughput Text Generation for LLMs via MII and DeepSpeed-Inference. *arXiv preprint arXiv:2401.08671* (2024).

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *29th ACM Symposium on Operating Systems Principles*.

[7] Rui Lai, Haotian Liu, Cheng Lu, Zhen Liu, Shiyi Cao, Shuo Shao, Yuxin Zhang, Luo Mai, and Dmitrii Ustiugov. 2025. TokenScale: Timely and Accurate Autoscaling for Disaggregated LLM Serving with Token Velocity. *arXiv preprint arXiv:2512.03416* (2025).

[8] Xueshen Liu, Yongji Wu, Yuncheng Yao, Danyang Zhuo, Ion Stoica, and Z. Morley Mao. 2026. Foundry: Template-Based CUDA Graph Context Materialization for Fast LLM Serving Cold Start. *arXiv preprint arXiv:2604.06664* (2026).

[9] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *arXiv preprint arXiv:2311.18677* (2023).

[10] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Wenguang Zheng, and Xin Xu. 2025. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. In *23rd USENIX Conference on File and Storage Technologies*.

[11] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *40th International Conference on Machine Learning*.

[12] Qiyang Su, Wenxuan Zhao, Xupeng Li, Madhav Andoorveedu, Chao Jiang, Zhe Zhu, Kaichen Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-Throughput LLM Inference via Model Re-Sharding. *arXiv preprint arXiv:2503.06433* (2025).

[13] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.

[15] Yuxin Wang, Yuhan Chen, Zeyu Li, Xin Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2024. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. *arXiv preprint arXiv:2401.17644* (2024).

[16] Bingyang Wu, Sheng Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. *arXiv preprint arXiv:2404.09526* (2024).

[17] Y. Xiang, X. Li, et al. 2025. Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market. In *31st ACM Symposium on Operating Systems Principles*.

[18] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*.

[19] Minchen Yu, Rui Yang, Chaobo Jia, Zhaoyuan Su, Sheng Yao, Tingfeng Lan, Yuchen Yang, Zirui Wang, Yue Cheng, Wei Wang, Ao Wang, and Ruichuan Chen. 2026. FaaScale: Unlocking Fast LLM Scaling for Serverless Inference. In *Proceedings of Machine Learning and Systems*.

[20] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*.

[21] Yinmin Zhong, Sheng Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*.